

Modélisation algorithmique complète

Optimisation des formations de coordonnateurs communaux - EAR 2027

Projet cc-formation-optimizer

25 juin 2026

Résumé

Ce document présente la modélisation algorithmique mise en oeuvre dans le projet `cc-formation-optimizer`. Il décrit le problème métier, les données d'entrée, les notations mathématiques, la formulation CP-SAT, le pipeline de résolution, les contrôles de validation et les exports produits. Il distingue explicitement les contraintes effectivement codées des alertes d'interprétation et des évolutions métier possibles.

Table des matières

1	Introduction	3
2	Description métier du problème	3
2.1	Communes, catégories et coordonnateurs	3
2.2	Sessions et pivots	3
2.3	Territoires EAR et lisibilité métier	4
2.4	Capacité, trajets et cohérence PC/TPC	4
3	Données d'entrée	4
3.1	Données communales	4
3.2	Données de trajet	4
3.3	Paramètres de configuration	5
3.4	Données dérivées	5
4	Notations mathématiques	5
4.1	Ensembles	5
4.2	Paramètres	5
4.3	Variables de décision	6
5	Formulation mathématique	6
5.1	Formulation synthétique	6
5.2	Lecture des contraintes	7

6	Fonction objectif	8
7	Méthode algorithmique	8
7.1	Rôle de CP-SAT	8
7.2	Pipeline de calcul	8
7.3	Faisabilité, optimalité et validation	9
8	Validation de la solution	9
9	Exports et interprétation	10
9.1	Fichiers produits	10
9.2	Indicateurs de qualité	10
9.3	Carte HTML	10
10	Limites et points métier à arbitrer	11
10.1	Le pivot doit-il appartenir à sa propre session ?	11
10.2	Accepte-t-on les sessions multi-territoires ?	11
10.3	Accepte-t-on des TPC dans des sessions PC ?	11
10.4	Faut-il pénaliser davantage les trajets longs ?	11
10.5	Faut-il viser moins de 55 sessions ?	11
11	Conclusion	11

1 Introduction

Le projet vise à organiser les sessions de formation des coordonnateurs communaux pour les communes PC et TPC de l'EAR 2027. Chaque commune doit être rattachée à une session, elle-même portée par une commune pivot. Une solution acceptable doit respecter des contraintes opérationnelles fortes : capacité en nombre de coordonnateurs communaux, temps de trajet maximal, budgets de formations PC et TPC, cohérence de type entre communes PC et sessions TPC, et existence effective des données de trajet.

Ce problème relève naturellement de l'optimisation sous contraintes. Il ne s'agit pas seulement de construire des groupes arbitraires de communes : il faut choisir quelles sessions ouvrir, décider à quel pivot rattacher chaque commune, satisfaire les contraintes dures et arbitrer entre plusieurs solutions faisables selon un objectif pondéré. Le modèle actuel est formulé comme un programme linéaire en variables entières et résolu avec OR-Tools CP-SAT.

L'objectif du projet n'est pas uniquement mathématique. Le solveur fournit une solution candidate, mais l'outil doit aussi produire une solution exploitable, lisible et contrôlable par les acteurs métier. C'est pourquoi la chaîne logicielle inclut une extraction métier, une validation indépendante, des exports détaillés, des indicateurs d'alerte et une carte HTML de contrôle.

2 Description métier du problème

2.1 Communes, catégories et coordonnateurs

L'unité de base du problème est la commune. Chaque commune possède un code INSEE, un nom, une population, une catégorie métier et éventuellement un territoire EAR, un nombre de logements et des coordonnées géographiques. Les catégories utilisées par le modèle sont PC et TPC.

Le modèle ne raisonne pas seulement en nombre de communes, mais en charge de formation. Cette charge est exprimée en nombre de coordonnateurs communaux à former. Une petite commune représente un coordonnateur, tandis qu'une commune dont la population dépasse le seuil configuré représente deux coordonnateurs. Cette règle donne plus de poids aux communes les plus peuplées dans les contraintes de capacité et dans le coût de trajet.

2.2 Sessions et pivots

Une session de formation est identifiée par une commune pivot et par un rang de slot. Toute commune chargée dans les données peut devenir pivot potentiel. Le nombre de slots dépend de la catégorie de la commune pivot : une commune PC peut porter jusqu'à trois sessions candidates, tandis qu'une commune TPC ne peut porter qu'une session candidate.

Une session candidate ne devient réelle que si la variable d'ouverture correspondante vaut 1. Le modèle actuel ne force pas une commune pivot ouverte à être affectée à la session qu'elle porte. Ce choix est important : il permet au solveur d'utiliser une commune comme lieu ou point d'organisation sans imposer que cette commune fasse partie de ce groupe, mais il peut produire des situations qui appellent un arbitrage métier.

2.3 Territoires EAR et lisibilité métier

Les territoires EAR sont chargés dans les données communales et apparaissent dans les exports. Ils servent à qualifier les groupes, à calculer un territoire majoritaire et à signaler des sessions multi-territoires ou des affectations à un pivot d'un autre territoire. En revanche, dans l'état actuel du code, le territoire EAR n'est pas une contrainte dure du modèle CP-SAT et n'intervient pas directement dans la fonction objectif.

2.4 Capacité, trajets et cohérence PC/TPC

Chaque session ouverte doit contenir une charge comprise entre un minimum L et une capacité maximale Q , exprimée en nombre de CC. Les trajets sont orientés : le temps de i vers j peut différer du temps de j vers i . Un couple commune-pivot est admissible seulement si le trajet existe dans les données préparées et si son temps est inférieur ou égal au seuil maximal T .

La cohérence PC/TPC est gérée par une contrainte dure : une commune PC ne peut pas être affectée à une session de type TPC. L'inverse n'est pas interdit : une commune TPC peut être affectée à une session PC, mais cette mixité est pénalisée dans l'objectif.

3 Données d'entrée

3.1 Données communales

Le solveur ne lit pas directement les fichiers bruts. La commande `prepare-data` produit des CSV propres à partir des fichiers ODS ou CSV du dossier brut. Le fichier communal propre contient notamment :

- le code commune ;
- le nom de la commune ;
- la catégorie PC ou TPC ;
- le territoire EAR 2027 ;
- la population ;
- le nombre de logements ;
- la latitude et la longitude si elles sont disponibles.

La règle de calcul du nombre de CC est :

$$q_i = \begin{cases} 1, & \text{si } p_i \leq p^*, \\ 2, & \text{si } p_i > p^*, \end{cases} \quad (1)$$

où p_i désigne la population de la commune i et où le seuil configuré est $p^* = 5000$.

3.2 Données de trajet

Les temps de trajet sont préparés dans un CSV contenant une origine, un pivot candidat et un temps en minutes. La préparation conserve l'orientation des trajets et ne complète pas les valeurs absentes. Cette règle est structurante : un trajet absent ne correspond pas à un coût très élevé, mais à une liaison interdite car aucune variable d'affectation ne sera créée pour ce couple.

3.3 Paramètres de configuration

Les paramètres métier sont centralisés dans `config/config_ear2027.yaml`. Les valeurs courantes les plus importantes sont :

Paramètre	Valeur courante	Rôle
T	75	temps maximal admissible en minutes
Q	14	capacité maximale d'une session en CC
L	6	remplissage minimal d'une session ouverte
B	55	budget total de sessions
f	45	budget de sessions PC
k	10	budget de sessions TPC
w_t	1	poids du trajet
w_e	1000	poids des coûts d'éligibilité
w_m	500	poids de la mixité résiduelle

La configuration impose aussi $B = f + k$, $M_{PC} = 3$, $M_{TPC} = 1$ et la règle de population de l'équation (1).

3.4 Données dérivées

À partir des CSV propres et du YAML, le code construit les ensembles, les variables candidates, les admissibilités, les compatibilités par défaut, les coûts d'éligibilité et le nombre de slots par pivot. Ces objets dérivés sont créés dans `parameters.py` et transmis au constructeur du modèle.

4 Notations mathématiques

4.1 Ensembles

Notation	Signification
C	ensemble des communes à affecter.
$P \subset C$	ensemble des communes de catégorie PC.
$R \subset C$	ensemble des communes de catégorie TPC. La lettre R est utilisée ici pour éviter la confusion avec le seuil de trajet T .
$F = C$	ensemble des communes candidates pivot.
M_j	nombre de slots candidats du pivot j , avec $M_j = 3$ si $j \in P$ et $M_j = 1$ si $j \in R$.
S	ensemble des sessions candidates (j, m) , où $j \in F$ et $m \in \{1, \dots, M_j\}$.
S_i	ensemble des sessions candidates pour lesquelles une variable d'affectation de la commune i existe effectivement.

4.2 Paramètres

Notation	Signification
p_i	population de la commune i .

Notation	Signification
q_i	nombre de CC associés à la commune i .
τ_{ij}	temps de trajet orienté de la commune i vers le pivot j .
T	seuil maximal admissible de trajet.
a_{ij}	indicateur d'admissibilité du trajet $i \rightarrow j$.
b_{ij}	indicateur de compatibilité métier du couple i, j .
Q	capacité maximale d'une session ouverte.
L	remplissage minimal d'une session ouverte.
B	budget total déclaré de sessions.
f	budget maximal de sessions PC.
k	budget maximal de sessions TPC.
e_j^{PC}	coût d'éligibilité du pivot j pour une session PC.
e_j^{TPC}	coût d'éligibilité du pivot j pour une session TPC.
w_t, w_e, w_m	poids respectifs du trajet, de l'éligibilité et de la mixité.

L'admissibilité et la compatibilité sont définies par :

$$a_{ij} = \begin{cases} 1, & \text{si le trajet } i \rightarrow j \text{ existe et } \tau_{ij} \leq T, \\ 0, & \text{sinon,} \end{cases} \quad (2)$$

$$b_{ij} \in \{0, 1\}. \quad (3)$$

En l'absence de fichier de compatibilités, le code pose $b_{ij} = 1$ pour tous les couples i, j .

4.3 Variables de décision

Les variables effectivement créées sont les suivantes :

Variable	Signification
$x_{ijm} \in \{0, 1\}$	vaut 1 si la commune i est affectée à la session (j, m) . Cette variable n'existe que si $a_{ij} = 1$ et $b_{ij} = 1$.
$y_{jm} \in \{0, 1\}$	vaut 1 si la session candidate (j, m) est ouverte.
$z_{jm} \in \{0, 1\}$	vaut 1 si la session ouverte (j, m) est de type TPC; si $y_{jm} = 1$ et $z_{jm} = 0$, elle est de type PC.
$d_{jm} \in \{0, \dots, Q\}$	variable entière représentant la mixité résiduelle, c'est-à-dire la charge TPC dans une session PC.

5 Formulation mathématique

5.1 Formulation synthétique

Le modèle minimise une somme pondérée de trois composantes :

$$\min \quad w_t O_{\text{trajet}} + w_e O_{\text{eligibilite}} + w_m O_{\text{mixite}} \quad (4)$$

avec :

$$O_{\text{trajet}} = \sum_{i \in C} \sum_{(j,m) \in S_i} q_i \tau_{ij} x_{ijm}, \quad (5)$$

$$O_{\text{eligibilite}} = \sum_{(j,m) \in S} \left[e_j^{\text{PC}} y_{jm} + \left(e_j^{\text{TPC}} - e_j^{\text{PC}} \right) z_{jm} \right], \quad (6)$$

$$O_{\text{mixite}} = \sum_{(j,m) \in S} d_{jm}. \quad (7)$$

Les contraintes du modèle actuel sont :

$$\sum_{(j,m) \in S_i} x_{ijm} = 1 \quad \forall i \in C, \quad (8)$$

$$x_{ijm} \leq y_{jm} \quad \forall i \in C, \forall (j,m) \in S_i, \quad (9)$$

$$L y_{jm} \leq \sum_{i \in C: (j,m) \in S_i} q_i x_{ijm} \quad \forall (j,m) \in S, \quad (10)$$

$$\sum_{i \in C: (j,m) \in S_i} q_i x_{ijm} \leq Q y_{jm} \quad \forall (j,m) \in S, \quad (11)$$

$$z_{jm} \leq y_{jm} \quad \forall (j,m) \in S, \quad (12)$$

$$\sum_{(j,m) \in S} (y_{jm} - z_{jm}) \leq f, \quad (13)$$

$$\sum_{(j,m) \in S} z_{jm} \leq k, \quad (14)$$

$$y_{j,m+1} \leq y_{jm} \quad \forall j \in P, \forall m \in \{1, \dots, M_j - 1\}, \quad (15)$$

$$x_{ijm} \leq 1 - z_{jm} \quad \forall i \in P, \forall (j,m) \in S_i, \quad (16)$$

$$d_{jm} \geq \sum_{i \in R: (j,m) \in S_i} q_i x_{ijm} - Q z_{jm} \quad \forall (j,m) \in S, \quad (17)$$

$$x_{ijm}, y_{jm}, z_{jm} \in \{0, 1\}, \quad d_{jm} \in \{0, \dots, Q\}. \quad (18)$$

Le budget total B est vérifié en configuration par l'invariant $B = f + k$. Le modèle CP-SAT ne contient pas une troisième contrainte séparée $\sum y_{jm} \leq B$, car les contraintes (13) et (14) pilotent directement les deux composantes du budget.

5.2 Lecture des contraintes

La contrainte (8) impose que chaque commune soit affectée exactement une fois. Si une commune ne dispose d'aucune session admissible, la somme est vide et le modèle devient infaisable, ce qui reflète un problème de données ou de paramétrage.

La contrainte (9) interdit d'affecter une commune à une session fermée. Les contraintes (10) et (11) encadrent la charge en CC de chaque session ouverte. Lorsqu'une session est fermée, $y_{jm} = 0$ force sa charge à zéro.

La contrainte (12) garantit qu'une session fermée ne peut pas être déclarée TPC. Les contraintes (13) et (14) limitent respectivement le nombre de sessions PC et TPC. L'expression $y_{jm} - z_{jm}$ vaut 1 pour une session PC ouverte, et 0 pour une session TPC ou fermée.

La contrainte (15) réduit les symétries pour les pivots PC : un slot de rang supérieur ne peut être ouvert que si les slots précédents le sont déjà. Cette règle ne change pas la nature des groupes possibles, mais elle stabilise et simplifie la recherche du solveur.

La contrainte (16) interdit les communes PC dans les sessions TPC. Elle porte sur la catégorie des communes affectées, pas sur la catégorie du pivot. Une session TPC peut donc être portée par un pivot PC si ce pivot n'est pas affecté à cette session.

La contrainte (17) définit la variable de mixité. Si la session est TPC, $z_{jm} = 1$, le terme $-Qz_{jm}$ rend la borne non contraignante grâce à la capacité maximale. Si la session est PC, $z_{jm} = 0$, et d_{jm} couvre la charge TPC présente dans cette session PC.

6 Fonction objectif

La fonction objectif (4) agrège trois critères réellement présents dans le code.

Le terme de trajet (5) mesure le coût total des déplacements. Chaque temps est pondéré par q_i , de sorte qu'une commune représentant deux coordonnateurs pèse deux fois plus qu'une commune représentant un seul coordonnateur.

Le terme d'éligibilité (6) favorise les pivots jugés plus adaptés selon les bandes de population configurées. La formulation est linéaire : une session PC paie e_j^{PC} , tandis qu'une session TPC paie e_j^{TPC} . Les coûts très élevés sont des pénalités, pas des interdictions dures. Un pivot reste mathématiquement possible si les contraintes l'autorisent, mais il devient très défavorable.

Le terme de mixité (7) pénalise les CC TPC affectés à des sessions PC. Le modèle autorise cette situation, car elle peut être nécessaire pour respecter les capacités, les budgets et les trajets. Elle est toutefois dégradée dans l'objectif afin d'être utilisée seulement si elle améliore l'équilibre global.

Le modèle actuel ne contient pas de coût fixe d'ouverture de session, de pénalité territoriale dure ou douce, de pénalité explicite pour pivot absent de sa session, ni de contrainte de superviseur, de calendrier ou de disponibilité. Ces éléments peuvent être étudiés comme extensions, mais ne doivent pas être présentés comme des composantes actuelles.

7 Méthode algorithmique

7.1 Rôle de CP-SAT

Le solveur utilisé est OR-Tools CP-SAT. Il manipule des variables booléennes et entières, des contraintes linéaires et une fonction objectif linéaire. Le modèle ne contient pas de produit entre variables : les expressions de type, de budget et d'éligibilité sont linéarisées directement.

La configuration transmet au solveur une limite de temps, un nombre de workers, une graine aléatoire et l'activation éventuelle des logs. Ces paramètres influencent la recherche, mais ne modifient pas le modèle mathématique.

7.2 Pipeline de calcul

La chaîne complète est volontairement séparée en étapes :

1. chargement de la configuration YAML ;
2. préparation des données brutes en CSV propres ;
3. chargement des communes, trajets et compatibilités ;
4. construction des paramètres dérivés ;

5. diagnostic pré-résolution ;
6. construction du modèle CP-SAT ;
7. résolution stricte avec `solve`, ou résolution assouplie avec `solve-relaxed` ;
8. extraction de la solution métier ;
9. validation indépendante ;
10. exports et carte HTML si demandés.

Le workflow `solve-relaxed` commence par la configuration initiale, puis teste une hiérarchie d'assouplissements explicitement configurés : modification du poids de mixité, réduction des coûts TPC, augmentation du seuil de trajet, réduction de L , augmentation de Q , augmentation cohérente de f , k et B , puis remplacement final de coûts très élevés par une pénalité finie. La contrainte PC vers session TPC n'est jamais relâchée automatiquement.

7.3 Faisabilité, optimalité et validation

Un statut **OPTIMAL** signifie que le solveur a trouvé une solution et prouvé qu'aucune meilleure solution n'existe pour le modèle donné. Un statut **FEASIBLE** signifie qu'une solution respectant les contraintes a été trouvée, mais que la preuve d'optimalité n'est pas acquise dans les limites de recherche.

Dans un contexte métier, une solution **FEASIBLE** peut être exploitable si elle passe la validation et si ses indicateurs sont acceptables. Elle ne doit cependant pas être décrite comme optimale. La validation métier reste distincte de la validation algorithmique : le code vérifie les contraintes codées, mais ne décide pas à la place des experts si une exception territoriale, un pivot absent ou une mixité résiduelle est acceptable.

8 Validation de la solution

Le solveur retourne des valeurs de variables. Le module `solution_extractor.py` reconstruit ensuite des sessions ouvertes, des affectations de communes et les composantes d'objectif. Cette solution extraite est contrôlée par `validation.py` avant tout export.

Les contrôles effectivement réalisés sont :

- chaque commune du modèle est affectée exactement une fois ;
- aucune affectation ne référence une session fermée ou inconnue ;
- chaque session respecte le remplissage minimal et la capacité maximale ;
- les budgets PC, TPC et total implicite sont respectés ;
- aucune commune PC n'est affectée à une session TPC ;
- chaque trajet utilisé existe et respecte le seuil T ;
- aucune compatibilité interdite n'est utilisée ;
- les types de session sont valides ;
- la variable de mixité est cohérente avec les affectations ;
- les composantes d'objectif et l'objectif total sont recalculés.

Les incohérences territoriales ne sont pas des violations bloquantes dans le code actuel. Elles sont signalées dans les exports comme alertes métier. De même, l'absence du pivot dans sa propre session n'est pas une violation algorithmique aujourd'hui.

9 Exports et interprétation

9.1 Fichiers produits

Lorsque la validation réussit, le pipeline peut produire :

- `sessions.csv`, avec une ligne par session ouverte ;
- `communes_affectees.csv`, avec une ligne par commune affectée ;
- `rapport_solution.md`, rapport lisible par un utilisateur métier ;
- `statistiques_solution.json`, synthèse structurée ;
- `config_utilisee.yaml`, copie de la configuration de résolution ;
- `solution_formation.xlsx`, si `openpyxl` est disponible ;
- `solution_map.html`, carte HTML autonome ;
- les journaux d’assouplissement si `solve-relaxed` est utilisé.

Les exports sont des produits de restitution. Ils ne remplacent pas la validation initiale : ils sont écrits seulement après que la solution extraite a été acceptée par `validate_solution()`.

9.2 Indicateurs de qualité

Les exports permettent de suivre plusieurs indicateurs utiles :

- nombre de sessions ouvertes et saturation des budgets ;
- nombre de sessions PC et TPC ;
- taux de remplissage et places restantes ;
- temps moyen et maximal ;
- sessions proches de la capacité maximale ;
- sessions faiblement remplies ;
- sessions multi-territoires ;
- mixité TPC dans les sessions PC ;
- pivots avec coût d’éligibilité élevé ;
- communes affectées à un pivot d’un autre territoire.

Ces indicateurs ne sont pas tous des contraintes. Ils servent à prioriser la revue humaine et à identifier les arbitrages possibles.

9.3 Carte HTML

La carte HTML est un outil de contrôle visuel. Elle embarque les statistiques globales, les contrôles de validation, les points cartographiés et une synthèse des sessions. Elle utilise les coordonnées latitude/longitude des communes et un fond IGN/Géoplateforme WMTS, tout en conservant les points visibles si le fond externe ne charge pas.

La commande `render-map` permet de régénérer la carte depuis des exports existants sans relancer le solveur. Cette distinction est importante lorsque la résolution réelle est longue.

10 Limites et points métier à arbitrer

Le modèle actuel est volontairement centré sur l'affectation des communes aux sessions. Plusieurs questions restent des choix métier, pas des erreurs du solveur.

10.1 Le pivot doit-il appartenir à sa propre session ?

Aujourd'hui, aucune contrainte n'impose qu'une commune pivot soit affectée à la session qu'elle porte. Imposer cette règle nécessiterait de garantir l'existence des trajets diagonaux $j \rightarrow j$ et d'ajouter une contrainte explicite. Selon le niveau de rigidité souhaité, cette règle pourrait être une contrainte dure ou une pénalité douce.

10.2 Accepte-t-on les sessions multi-territoires ?

Les territoires EAR sont visibles dans les exports, mais ne contraignent pas le solveur. Si la lisibilité territoriale devient prioritaire, il faudrait ajouter une pénalité ou une contrainte dédiée. Ce choix peut augmenter le coût de trajet ou rendre certaines zones plus difficiles à affecter.

10.3 Accepte-t-on des TPC dans des sessions PC ?

Les TPC dans des sessions PC sont autorisées et pénalisées. À l'inverse, les PC dans des sessions TPC sont interdites. Si la mixité résiduelle est jugée trop élevée, il est possible de calibrer w_m , d'ajuster les budgets TPC ou de tester une contrainte plus stricte.

10.4 Faut-il pénaliser davantage les trajets longs ?

Le modèle interdit les trajets au-delà de T et minimise la somme pondérée des temps admissibles. Il ne contient pas de pénalité spécifique pour les trajets proches du seuil, en dehors de leur contribution linéaire au coût de trajet. Une pénalité par palier pourrait être envisagée si les trajets proches de T sont jugés trop nombreux.

10.5 Faut-il viser moins de 55 sessions ?

Le modèle respecte les plafonds f , k et B , mais ne pénalise pas l'ouverture d'une session en tant que telle. Avec les paramètres actuels, une solution peut donc saturer le plafond de 55 sessions si cela améliore les autres termes de l'objectif. Si l'objectif métier est de réduire le nombre de sessions, il faut ajouter un coût d'ouverture ou revoir les paramètres de capacité et de remplissage.

11 Conclusion

Le modèle fournit une base reproductible et traçable pour organiser les formations des coordonnateurs communaux. Il transforme des données brutes en tables propres, construit une formulation CP-SAT cohérente avec les règles métier codées, produit une solution candidate, puis la valide et l'exporte dans des formats exploitables.

Son intérêt principal est de rendre explicites les arbitrages : temps de trajet, capacité, budgets, éligibilité des pivots et mixité PC/TPC. L'outil ne remplace pas l'arbitrage métier. Il fournit une

solution contrôlée, documentée et visualisable, ainsi qu'une base solide pour tester des évolutions de modèle si les règles métier évoluent.